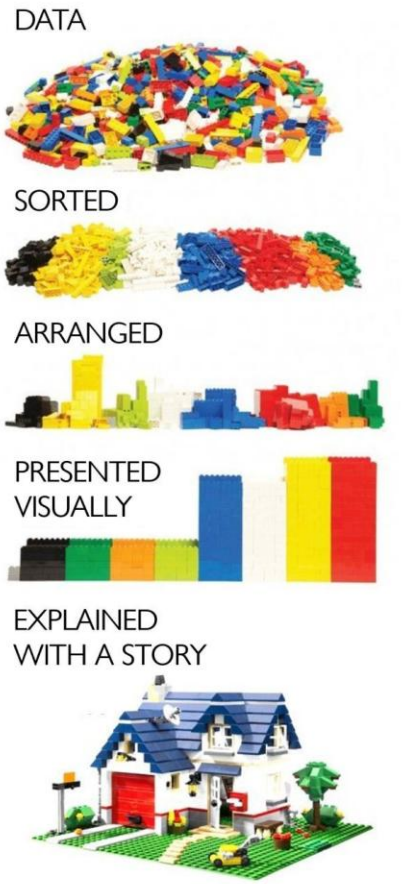
ADS1 - Curs 2
Analiza exploratorie a datelor

Instructor: Conf. Dr. Teodor-Ștefan Bîldea
SIIPA, An I, Semestrul I



Agendă

Analiza exploratorie (EDA)



Data Mining <- Matematică

Laureat al medaliei Fields in 2010- Cedric Villani - despre matematica (min. 7-10)

Analiza Exploratorie (EDA)

Analiza cantitativă și
Preprocesare



Intro

Exploratory Data Analysis - EDA: *John Tukey* - *1977*

> “abilitatea de a prelua datele - de a le înțelege, procesa, extrage valoare din ele, de a le vizualiza, de a comunica informația - acestea toate o să fie un bagaj de cunoștințe extrem de important în următoarele decade, [...] deoarece acum avem peste tot date in mod gratuit. Deci partea complementară mai puțin frecventă decât datele este abilitatea de a le înțelege și a extrage valoare din ele.”

Hal Varian, Google’s Chief Economist
The McKinsey Quarterly, Jan 2009

Analiza cantitativă

- În general, datele trebuie preprocesate pentru a le putea folosi
- Pași de preprocesare:
  - Determinarea gradului de umplere a coloanelor
    - Calculul procentului de valori lipsă
  - Determinarea liniilor cu date lipsă
    - Numărul liniilor care au valori lipsă
- Decizii de luat (modelele nu acceptă date cu valori lipsă!):
  - Imputarea de valori de pe linii și/sau coloane
  - Eliminarea coloanelor cu prea multe valori lipsă
  - Eliminarea înregistrărilor care au câmpuri lipsă
  - Eliminarea coloanelor aproape constante (constant + date lipsă, sau constant 95%)

```
RangeIndex: 129971 entries, 0 to 129970
Data columns (total 13 columns):
 #   Column                 Non-Null Count   Dtype
---  ------                 --------------   -----
 0   points                 129971 non-null  int64
 1   title                  129971 non-null  object
 2   description            129971 non-null  object
 3   taster_name            103727 non-null  object
 4   taster_twitter_handle  98758 non-null   object
 5   price                  120975 non-null  float64
 6   designation            92506 non-null   object
 7   variety                129970 non-null  object
 8   region_1               108724 non-null  object
 9   region_2               50511 non-null   object
 10  province               129908 non-null  object
 11  country                129908 non-null  object
 12  winery                 129971 non-null  object
```

Exemple de coloane inutile

- În practică nu prea avem ce face cu coloane care au o gaură de peste un numit prag procentual – de exemplu 20%
  - Putem încerca imputarea valorilor nule sau creearea unei valori speciale pentru datele speciale, dar depinde de la caz la caz în funcție de cât de informativă e coloana
- Dacă avem date suficiente, eliminarea înregistrărilor cu date lipsă este cea mai sigură opțiune
  - Alternativa cu păstrarea și fie creare de valori dedicate, fie imputare din înregistrări similare ar trebui testată – în definitiv, ceea ce contează este ca odată pus în producție modelul să genereze valoare

Imputarea datelor

Metoda	Descriere
Imputare cu valoarea medie	Înlocuirea valorilor lipsă cu media sau media ponderată a caracteristicii. Poate simplifica datele, dar poate introduce distorsiuni.
Imputare cu valoarea medianei	Înlocuirea valorilor lipsă cu mediana caracteristicii. Este mai robustă față de valori extreme (outliers).
Imputare cu valoarea modală	Înlocuirea valorilor lipsă cu valoarea modală, cea mai frecventă valoare din caracteristică, utilizată pentru date categorice sau nominale.
Imputare cu regresie	Construirea unui model de regresie pentru a prezice valorile lipsă pe baza altor caracteristici cunoscute. Oferă estimări mai precise, dar necesită mai mult efort de analiză.
Imputare cu vecini asemănători (K-NN)	Identificarea celor mai apropiați K vecini și înlocuirea valorilor lipsă cu o medie ponderată a valorilor acestor vecini. Utilă pentru date cu structuri complexe sau multe caracteristici.

Imputarea datelor

- Resurse Python

`sklearn.impute`: Impute

Transformers for missing value imputation

User guide: See the Imputation of missing values section for further details.

`impute.SimpleImputer`(*[, missing_values, ...])	Univariate imputer for completing missing values with simple strategies.
`impute.IterativeImputer`([estimator, ...])	Multivariate imputer that estimates each feature from all the others.
`impute.MissingIndicator`(*[, missing_values, ...])	Binary indicators for missing values.
`impute.KNNImputer`(*[, missing_values, ...])	Imputation for completing missing values using k-Nearest Neighbors.

Discretizarea valorilor numerice/continue

Beneficiu potențial	**Explicație**
Simplificarea complexității	Discretizarea face datele continue mai ușor de înțeles și gestionat.
Facilitarea analizei statistice	Multe tehnici statistice funcționează mai bine cu date discrete.
Reducerea dimensiunii datelor	Discretizarea reduce necesitatea de a stoca și procesa multe valori.
Compatibilitate	Anumiți algoritmi necesită date discrete pentru funcționare.
Interpretare	Categoriile sau intervalele sunt mai ușor de interpretat.
Evaluarea tendințelor	Discretizarea poate scoate în evidență tipare dificil de observat.
Reducerea sensibilității la zgomot	Datele discrete pot fi mai stabile în fața variabilității.

Discretizarea variabilelor numerice/continue

- Discretizare directă – stabilirea unor praguri după o euristică dictată de business sau de experiență
  - Exemplu: grupe de vârstă, de interes economic; grupe de vechime în muncă; grupe de salarizare; grupe de venit etc.
- Întrebări care trebuie adresate:
  - Cum ar trebui să stabilim intervalele pentru grupe? (grupe uniforme? Intervale stabilite prin lege sau prin reguli de business?)
  - Câte grupe sunt optime?

- În absența oricărei constrângeri, cum ați face?

Discretizarea variabilelor numerice/continue

- Discretizare cu coloană de referință
  - Problema: Gruparea salariaților după vârstă în 3 grupuri egale din punct de vedere al costurilor salariale: fiecare interval să reprezinte ~33% din costurile cu salariul

Grupare vârstă	Interval	Propoție total salarii
Grupa 1	??	33.33% din Total Salarii
Grupa 2	??	33.33% din Total Salarii
Grupa 3	??	33.33% din Total Salarii

ID Salariat	Salariu	Vârstă
1	7000	57
2	8000	61
3	6000	52
4	3000	61
5	5000	43
6	9000	63
7	4000	63
8	9000	52
9	2000	62
10	4000	33
11	10000	44
12	4000	36
13	2000	38
14	9000	37
15	7000	37

Discretizare cu coloană de referință

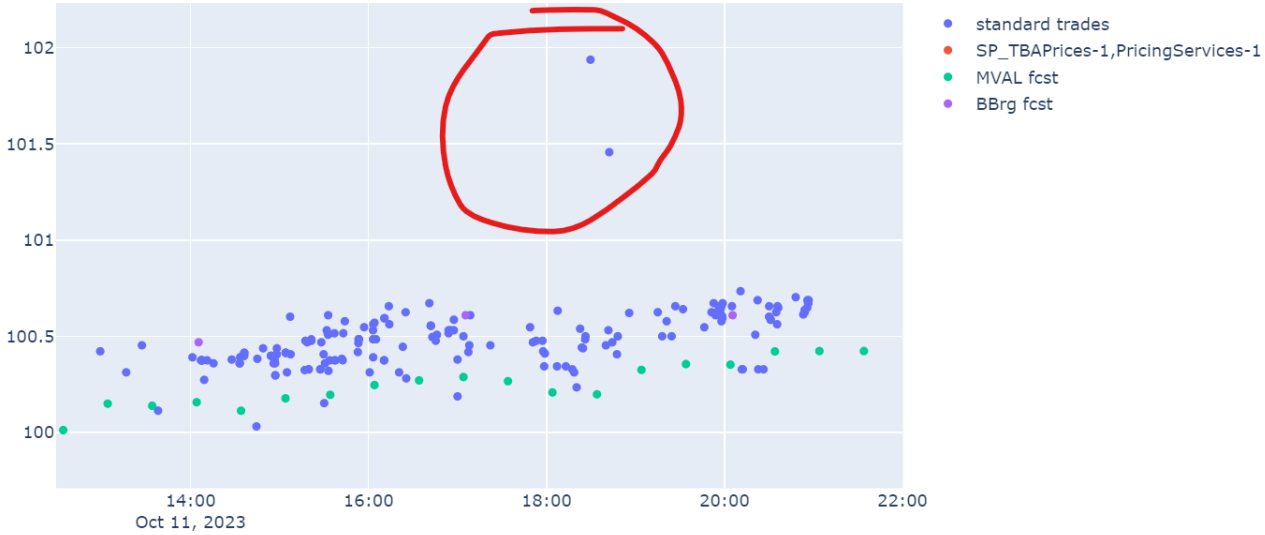
- Sortăm datele după coloana din care se formeaza grupele, crescător
- Calculăm suma cumulativă procentuală cumulată a salariilor
- Extragem limitele de vârstă aproximativ la percentilele 33, 67.

ID Salariat	Funcție	Salariu	Vârstă	Cost Cumulativ Si	Procentua	Grupare vârstă
10	Programator I	4000	33	4000	4%	Grupa 1
12	Programator I	4000	36	8000	9%	Grupa 1
14	CTO	9000	37	17000	19%	Grupa 1
15	Arhitect	7000	37	24000	27%	Grupa 1
13	Analist Junior	2000	38	26000	29%	Grupa 1
5	Șef echipă programatori	5000	43	31000	35%	Grupa 2
11	CEO	10000	44	41000	46%	Grupa 2
3	Director R&D	6000	52	47000	53%	Grupa 2
8	VIP Legal	9000	52	56000	63%	Grupa 2
1	Director Resurse Umane	7000	57	63000	71%	Grupa 2
2	Director Marketing	8000	61	71000	80%	Grupa 3
4	Contabil	3000	61	74000	83%	Grupa 3
9	Contabil	2000	62	76000	85%	Grupa 3
6	Director Contabilitate	9000	63	85000	96%	Grupa 3
7	Asistent CEO	4000	63	89000	100%	Grupa 3

Grupare vârstă	Interval	Propoție total salarii
Grupa 1	<= 40	33.33% din Total Salarii
Grupa 2	între 40 și 60	33.33% din Total Salarii
Grupa 3	>60	33.33% din Total Salarii

Anomalii

- Analiza anomaliilor (outliers) (asta doar în cazul în care problema nu constă tocmai în determinarea acestora... *outlier detection*)



Codificarea numerică a variabilelor categoriale

- Modelele statistice nu procesează decât numere
- Orice atribut categorial – geografice, culori, caracteristici – trebuie codificat

Culoare mașină	Codificare 1	Codificare 2	Codifcare oareacare
Alb	0	1	7
Albastru	1	0	-1
Roșu	2	2	2013

- Cum alegem o codifcare nepărtinitoare și care să nu fie aleatoare?

Codificarea numerică a variabilelor categoriale

- ONE HOT ENCODING – codificare nepărtinitoare

Culoare mașină
Alb
Albastru
Roșu

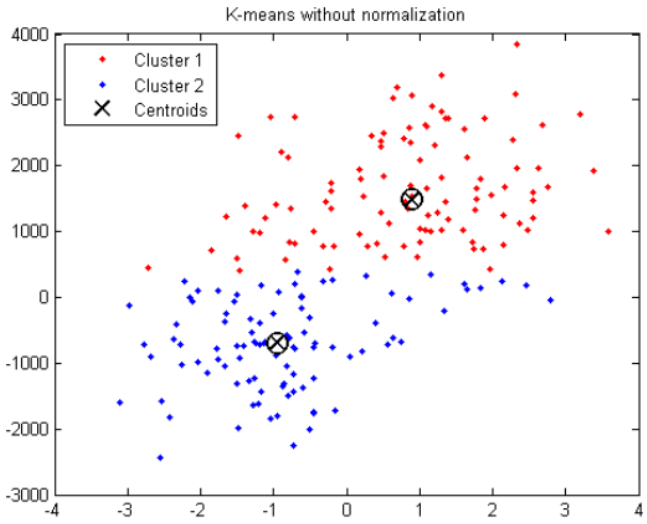


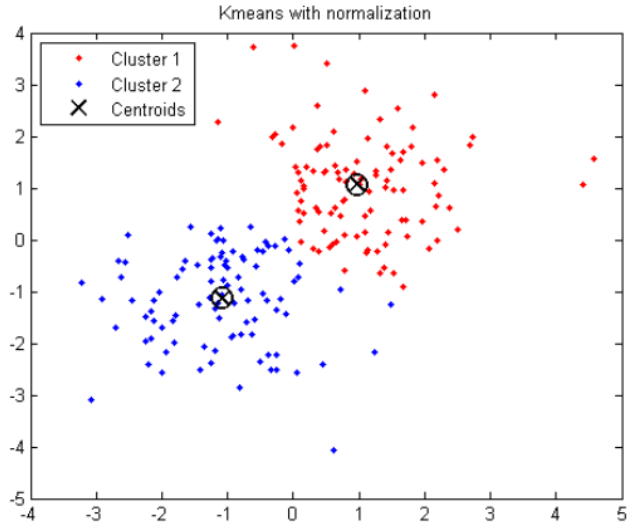
Culoare_Alb	Culoare_Abastru	Culoare_Roșu
1	0	0
0	1	0
0	0	1

- Alb = (1,0,0), Albastru = (0,1,0), Roșu = (0,0,1)
- Dezavantaj: în loc de o coloană vom avea 3 (=numărul de valori distincte pentru culoare); în plus toate culorile noi vor avea vectorul (0,0,0) care nu apare la antrenament.
- Pentru un atribut cu *n* valori distincte vom construi *n* coloane
- ATENTIE: Numărul de coloane va crește rapid dacă sunt prezente mai multe variabile categoriale cu multe valori distincte fiecare

Normalizare, standardizare (ref)

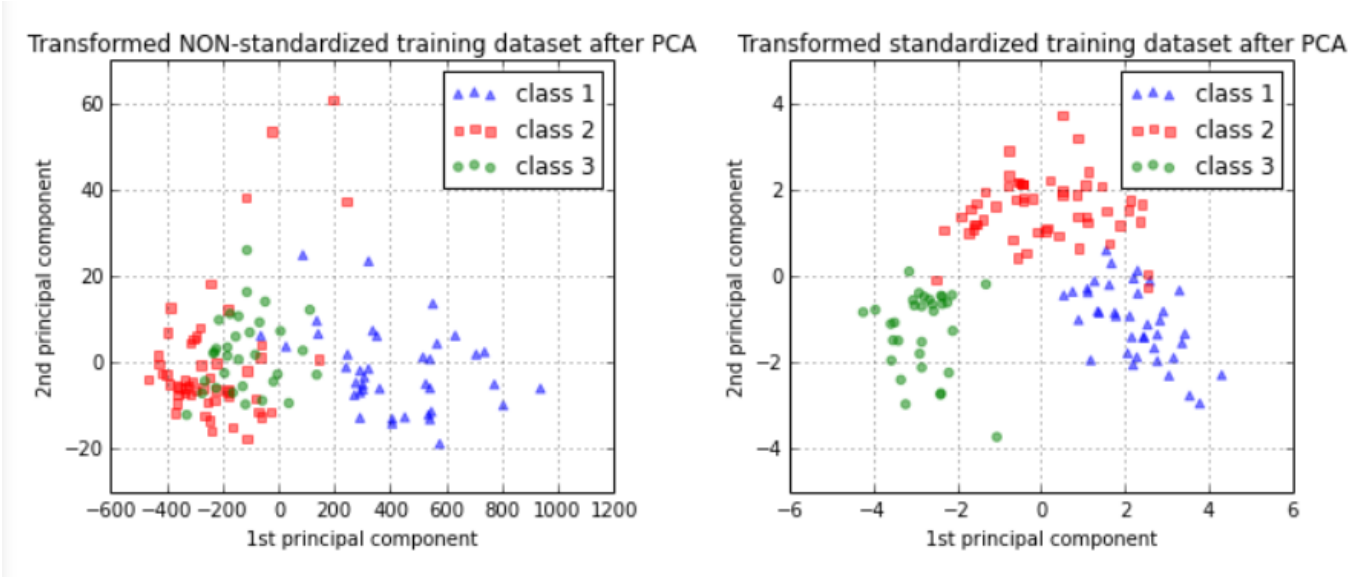
- Pentru modelele bazate pe distanță, standardizarea este efectuată pentru a preveni caracteristicile cu intervale mai largi să domine metrica distanței.





Normalizare, standardizare

- Metode de a comasa atributele numerice pe aceeiași scală pentru a putea măsura mai bine importanța lor, ca în cazul PCA, sau al regresiei statistice.



Normalizare, standardizare

- Motivul pentru care standardizăm datele nu este același pentru toate modelele de învățare automată și diferă de la un model la altul.

Beneficiu	Descriere
Compararea și interpretarea mai ușoară	Standardizarea transformă datele pentru a avea o medie de 0 și o deviație standard de 1, facilitând compararea și interpretarea.
Eliminarea unităților de măsură	Standardizarea elimină efectele unităților de măsură și dimensiunile diferite ale caracteristicilor, permitând compararea directă.
Îmbunătățirea performanței algoritmilor	Algoritmii de învățare automată sunt adesea influențați de scala datelor, iar standardizarea îmbunătățește performanța și convergența acestora.
Reducerea sensibilității la valori extreme	Standardizarea face datele mai robuste la valori extreme, evitând efectele incorecte ale acestora asupra analizei.
Simplificarea interpretării coeficienților	În modelele statistice, standardizarea facilitează interpretarea coeficienților, care devin mai direct interpretabili.

Normalizare, standardizare

- Aceste metode de standardizare pot fi aplicate în funcție de natura datelor, cerințele analizei și tipul de algoritm pe care intenționați să îl utilizați.
- Fiecare metodă are avantaje și dezavantaje și trebuie aleasă în funcție de contextul specific al setului de date și obiectivele analizei.

Metoda de Standardizare	Descriere
Standardizarea Z-score	Transformă datele astfel încât să aibă o medie de 0 și o deviație standard de 1, astfel încât valorile să fie centrate și scalate.
Min-Max Scaling	Redimensionează datele pentru a se încadra într-un interval specific, cum ar fi [0, 1], păstrând relațiile relative între valorile initiale.
Robust Scaling (Scaling robust)	Similar cu Standardizarea Z-score, dar utilizează statisticile robuste (mediană și cuartile) pentru a face față valorilor extreme.
Log Transformation	Aplică logaritmul natural asupra datelor, reducând variația în cazul datelor cu distribuție asimetrică sau exponențială.
Box-Cox Transformation	O metodă statistică care transformă datele într-o distribuție aproape normală, potrivită pentru date pozitive și negative.
Unit Vector Scaling	Redimensionează datele astfel încât vectorii caracteristicilor să aibă norma unitară, folosită în special în algoritmi care depind de distanță sau similaritate.

Normalizare, standardizare

- Resurse Python

The `sklearn.preprocessing` module includes scaling, centering, normalization, binarization methods.

User guide: See the Preprocessing data section for further details.

`preprocessing.Binarizer`(*[, threshold, copy])	Binarize data (set feature values to 0 or 1) according to a threshold.
`preprocessing.FunctionTransformer`([func, ...])	Constructs a transformer from an arbitrary callable.
`preprocessing.KBinsDiscretizer`([n_bins, ...])	Bin continuous data into intervals.
`preprocessing.KernelCenterer`()	Center an arbitrary kernel matrix K.
`preprocessing.LabelBinarizer`(*[, neg_label, ...])	Binarize labels in a one-vs-all fashion.
`preprocessing.LabelEncoder`()	Encode target labels with value between 0 and n_classes-1.
`preprocessing.MultiLabelBinarizer`(*[, ...])	Transform between iterable of iterables and a multilabel format.
`preprocessing.MaxAbsScaler`(*[, copy])	Scale each feature by its maximum absolute value.
`preprocessing.MinMaxScaler`([feature_range, ...])	Transform features by scaling each feature to a given range.
`preprocessing.Normalizer`([norm, copy])	Normalize samples individually to unit norm.
`preprocessing.OneHotEncoder`(*[, categories, ...])	Encode categorical features as a one-hot numeric array.
`preprocessing.OrdinalEncoder`(*[, ...])	Encode categorical features as an integer array.
`preprocessing.PolynomialFeatures`([degree, ...])	Generate polynomial and interaction features.
`preprocessing.PowerTransformer`([method, ...])	Apply a power transform featurewise to make data more Gaussian-like.
`preprocessing.QuantileTransformer`(*[, ...])	Transform features using quantiles information.
`preprocessing.RobustScaler`(*[, ...])	Scale features using statistics that are robust to outliers.
`preprocessing.SplineTransformer`([n_knots, ...])	Generate univariate B-spline bases for features.
`preprocessing.StandardScaler`(*[, copy, ...])	Standardize features by removing the mean and scaling to unit variance.
`preprocessing.TargetEncoder`([categories, ...])	Target Encoder for regression and classification targets.

Când este importantă standardizarea variabilelor?

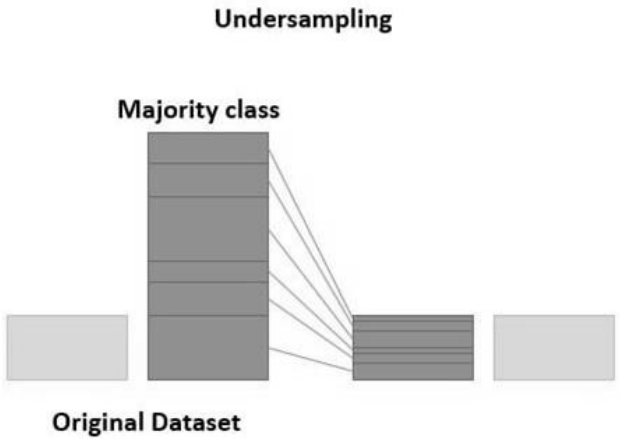
- Înainte de o clusterizare
  - Tehnicile de analiză a grupurilor depind de conceptul de măsurare a distanței dintre diferitele observații pe care încercăm să le grupăm. Dacă o variabilă este măsurată la o scară mai mare decât celelalte variabile, atunci orice măsură pe care o folosim va fi influențată excesiv de acea variabilă.
- Înainte de PCA
  - PCA oferă mai multă pondere acelor variabile care au variațe mai mari decât variabilelor care au variațe foarte mici. De fapt, rezultatele analizei vor depinde de ce unități de măsură sunt utilizate pentru măsurarea fiecărei variabile. Standardizarea valorilor brute face o variație egală, astfel încât ponderea mare nu este atribuită variabilelor care au variațe mai mari.
- Înainte de regresii (LASSO, RIDGE) sau când interpretăm o regresie clasică

Balansarea claselor (pt. clasificare)

- Dacă o clasă este dominantă (de exemplu mai mult de 90% din observații), devin greu de construit un model de clasificare
- În mod ideal, când antrenăm un model de clasificare cu n clase, datele de antrenament ar trebui să conțină un număr similar de observații pentru fiecare clasă
- În realitate nu se întâmplă așa. Uneori este benefică balansarea claselor
  - Prin supraeșantionare (oversampling), sau
  - Prin subeșantionare (undersampling)

Balansarea claselor prin subeșantionare (undersampling)

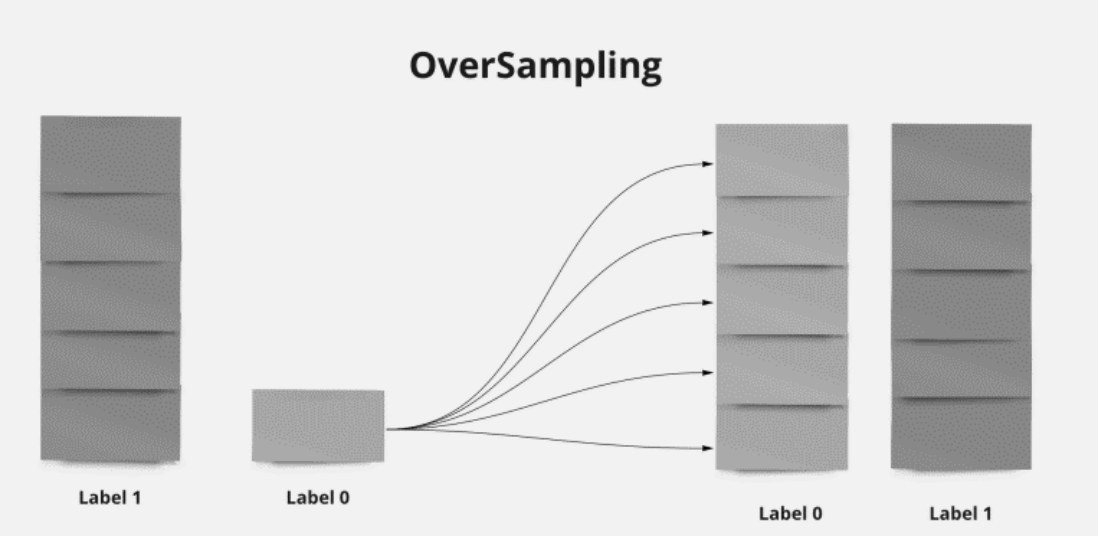
- Cea mai simplă strategie – din clasa mare se eșantionează de mărimea clasei mai mici
- Funcționează bine pentru Big Data



- Pentru date puține (ca în psihologie) nu este benefică
  - De exemplu dacă avem 100 de observații cu clasele 0 și 1 și 5 sunt cu clasa 0 restul clasa 1, subeșantionarea ne dă un set cu înregistrări – 5 cu clasa 1 și 5 cu clasa 0. Cam greu de făcut statistică cu 10 numere –contraexemplu:

Balansarea claselor prin supraeșantionare (oversampling)

- Strategia pornește cu clasa mai mică
- Se generează sintetic observații asemănătoare celor din clasa mică până când mărimea eșantionului este similară cu cea a clasei mari



Balansarea claselor – metode Python

- Resurse Python
  - https://imbalanced-learn.org/stable/
  - https://learn-scikit.oneoffcoder.com/imbalanced-learn.html

This is the full API documentation of the `imbalanced-learn` toolbox.

Under-sampling methods
- Prototype generation
  - ClusterCentroids
- Prototype selection
  - CondensedNearestNeighbour
  - EditedNearestNeighbours
  - RepeatedEditedNearestNeighbours
  - AllKNN
  - InstanceHardnessThreshold
  - NearMiss
  - NeighbourhoodCleaningRule
  - OneSidedSelection
  - RandomUnderSampler
  - TomekLinks

Over-sampling methods
- Basic over-sampling
  - RandomOverSampler
- SMOTE algorithms
  - SMOTE
  - SMOTENC
  - SMOTEN
  - ADASYN
  - BorderlineSMOTE
  - KMeansSMOTE
  - SVMSMOTE

Ingineria atributelor (Feature engineering)

- Constă în adăugarea de atribute derivate din datele existente
  - Atributele obținute prin discretizare
  - Atribute agregate obținute la nivel de grupări detectate (exemplu: volum vânzări clienți mici,medii mari)
  - Atribute obținute prin calcul direct, stabilind astfel legături complexe între atribute
    - Țin în general de natura problemei și depinde doar de creativitatea echipei pentru a inventa atribute care să aducă valoare

Exemplu - indexul de profit al unui produs în grupa din care face parte:

- se împarte profitul (mediu al) produsului la media de profit a produselor grupei
- astfel se poate stabili gradul de profitabilitate în categorie

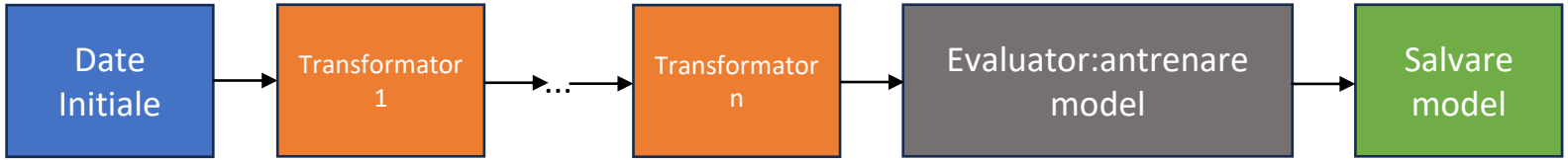
Produs	Grupa Produs	Marja Profit	Total Vânzări	IndexProfit
P1	G1	0.23	400000	0.3
P2	G1	0.47	900000	1.6
P3	G1	0.76	400000	1.1
P4	G1	0.84	300000	0.9
P5	G2	0.96	700000	2.2
P6	G2	0.08	600000	0.2
P7	G2	0.58	900000	1.7
P8	G2	0.45	500000	0.7
P9	G2	0.35	200000	0.2

Linii de procesare a datelor

- În general paradigma învățării automate constă în manipularea datelor prin înlănțuirea unor pași
- Pașii se pot împărți în:
  - Pași de transformare a datelor – setul de date de la intrare este prelucrat și la ieșire se obține un alt set de date
    - Exemple: filtrare, eliminare date lipsă, eliminare anomalii, imputare, codificarea datelor, standardizare, discretizare, ingineria atributelor
    - Acești pași sunt executați de transformatoare de date – module dedicate care execută pașii necesari
  - Pasul de antrenare/evaluare a unui model – setul de date de la intrare este folosit pentru a antrena un model de învățare automată, iar la ieșire avem un model care poate fi salvat și reutilizat pe date noi
    - Este un singur astfel de pas, ultimul de pe linia de procesare

Linii de procesare a datelor

- Linii de asamblare:



- Resurse Python:

sklearn.pipeline.Pipeline

class `sklearn.pipeline.Pipeline`(*steps*, *, *memory=None*, *verbose=False*) [source]

Pipeline of transforms with a final estimator.

Sequentially apply a list of transforms and a final estimator. Intermediate steps of the pipeline must be 'transforms', that is, they must implement `fit` and `transform` methods. The final estimator only needs to implement `fit`. The transformers in the pipeline can be cached using `memory` argument.

The purpose of the pipeline is to assemble several steps that can be cross-validated together while setting different parameters. For this, it enables setting parameters of the various steps using their names and the parameter name separated by a `'__'`, as in the example below. A step's estimator may be replaced entirely by setting the parameter with its name to another estimator, or a transformer removed by setting it to `'passthrough'` or `None`.